



UNIVERSITÀ DEGLI STUDI DI SALERNO

Dipartimento di Informatica

Corso di Laurea Triennale in Informatica

PROGETTO PROGRAMMAZIONE SICURA

Vulnerabilità di Contact Discovery in Apple AirDrop

PROFESSORESSA

Prof.ssa Barbara Masucci

Università degli Studi di Salerno

CANDIDATO

Costantino Paciello

Matricola: 0522502045

Vittorio Guida

Matricola: 0522502008

Abstract

Questo elaborato analizza una vulnerabilità di *Contact Discovery* nel protocollo AirDrop di Apple, mostrando come l'uso di hash SHA-256 senza salt su numeri di telefono a bassa entropia consenta a un attaccante di risalire all'identità degli utenti. La vulnerabilità viene studiata attraverso tre scenari di attacco distinti: *Sender Leakage*, *Receiver Leakage* e *Log File Leakage*. Per ciascun caso vengono descritte le condizioni necessarie, le modalità di sfruttamento e le possibili conseguenze sulla riservatezza degli utenti.

L'analisi mette in evidenza che il problema non deriva da un semplice errore di implementazione, ma da una scelta progettuale insicura, in cui una funzione crittografica viene usata per proteggere dati facilmente invertibili. Viene inoltre presentata la soluzione proposta dai ricercatori, *PrivateDrop*, basata su *Private Set Intersection*, che elimina la necessità di scambiare hash invertibili e preserva la funzionalità del servizio senza compromettere l'esperienza d'uso. Infine, l'elaborato discute la risposta parziale e selettiva di Apple, evidenziando il divario tra la correzione tecnica della vulnerabilità e le scelte effettivamente adottate nel prodotto.

Indice

Elenco delle Figure	v
Elenco delle Tabelle	vi
1 Introduzione	1
1.1 Contesto e motivazione	1
1.2 Il problema	2
1.3 Obiettivi del progetto	2
1.4 Struttura del documento	2
2 Il protocollo AirDrop	4
2.1 Panoramica generale	4
2.2 Identificatori di contatto e rubrica	4
2.3 Impostazioni di visibilità	5
2.4 Flusso completo del protocollo	5
2.4.1 Fase 1: Discovery	6
2.4.2 Fase 2: Authentication	7
2.4.3 Fase 3: Data Transfer	7
2.5 Il meccanismo di autenticazione reciproca	7
2.5.1 Struttura del Validation Record	7
2.5.2 Verifica dell'autenticità	8

2.5.3	Normalizzazione dei numeri di telefono	8
3	Il fallimento crittografico	9
3.1	Premessa: quando SHA-256 non basta	9
3.2	Bassa entropia degli identificatori di contatto	10
3.2.1	Numeri di telefono	10
3.2.2	Indirizzi e-mail	10
3.3	L'assenza di salt	11
3.4	Attacchi di inversione: rainbow table	11
3.4.1	Precomputazione	11
3.4.2	Tempo di inversione	12
3.5	Riepilogo delle debolezze	12
4	Modello di minaccia e albero di attacco	13
4.1	Obiettivo dell'attaccante	13
4.2	Modello di minaccia	13
4.3	Costruzione dell'albero di attacco	14
4.4	Ramo Sender Leakage	15
4.5	Ramo Receiver Leakage	16
4.6	Ramo Log File Leakage	16
4.7	Etichettatura dei nodi	17
4.8	Collegamento alle debolezze crittografiche	18
5	Le tre vulnerabilità	19
5.1	Sender Leakage	19
5.1.1	Descrizione	19
5.1.2	Meccanismo di attacco	20
5.1.3	Impatto	20
5.2	Receiver Leakage	20
5.2.1	Descrizione	20
5.2.2	Percorso di attacco	21
5.2.3	Scenario pratico	21
5.3	Log File Leakage	21

5.3.1	Descrizione	21
5.3.2	Percorso di attacco	22
5.3.3	Caso documentato	22
5.4	Confronto delle vulnerabilità	23
6	AirCollect: proof of concept	24
6.1	Introduzione ad AirCollect	24
6.2	Architettura del sistema	24
6.3	Implementazione tecnica	25
6.3.1	Generazione delle rainbow table	25
6.3.2	Cattura del traffico	25
6.3.3	Inversione degli hash	26
6.4	Risultati sperimentali	26
6.4.1	Efficacia dell'inversione	26
6.4.2	Test in scenario Sender Leakage	26
6.4.3	Test in scenario Receiver Leakage	27
6.5	Video dimostrativo	27
6.6	Implicazioni pratiche	27
6.7	Conclusione	28
7	Verifica Sperimentale della Vulnerabilità	29
7.1	Esperimento 1: Intercettazione con Honeypot e Leak dei Dati	29
7.2	Esperimento 2: Manipolazione degli Intenti (Spoofing URL)	31
7.3	Esperimento 3: Evoluzione dell'Attacco nelle Versioni Recenti di iOS	32
7.4	Sintesi dei Risultati	33
8	PrivateDrop: la soluzione	34
8.1	Il problema da risolvere	34
8.2	Private Set Intersection	35
8.2.1	Cos'è la PSI	35
8.2.2	PSI nel mondo reale	35
8.3	Il protocollo PrivateDrop	35
8.3.1	Le opzioni di design	35

8.3.2	La combinazione scelta: DO2 seguito da DO3	36
8.3.3	Come funziona la crittografia	37
8.3.4	Protezione contro input malevoli	37
8.4	Ottimizzazioni di performance	38
8.4.1	Precomputazione	38
8.4.2	Riduzione dei messaggi	38
8.4.3	Padding degli input	38
8.5	Valutazione delle performance	38
8.6	Integrazione nel protocollo AirDrop	39
8.7	Risposta di Apple	39
9	Conclusioni	40

Elenco delle figure

2.1	Protocollo AirDrop (versione semplificata). Le parti del messaggio evidenziate in arancione rivelano gli identificatori di contatto del mittente e del destinatario [1].	6
4.1	Albero di attacco.	15
4.2	Sender Leakage.	15
4.3	Receiver Leakage.	16
4.4	Log File Leakage.	17

Elenco delle tabelle

5.1	Confronto delle tre vulnerabilità di AirDrop	23
7.1	Evoluzione del modello di attacco AirDrop nelle versioni recenti di iOS.	33
8.1	Opzioni di design per l'applicazione di PSI ad AirDrop [1]	36
8.2	Confronto dei ritardi di autenticazione [1]	39

CAPITOLO 1

Introduzione

1.1 Contesto e motivazione

Negli ultimi anni la condivisione di file tra dispositivi mobili è diventata una funzionalità quotidiana per centinaia di milioni di utenti. Apple AirDrop, integrato in tutti i dispositivi iOS e macOS, è uno dei servizi di condivisione *peer-to-peer* più diffusi al mondo, con oltre 1,5 miliardi di dispositivi compatibili [1]. La semplicità d'uso e l'assenza di requisiti di connettività esterna lo rendono uno strumento capillare, impiegato quotidianamente in contesti pubblici quali metropolitane, aeroporti, università e luoghi di lavoro.

Proprio la natura pubblica di questi ambienti rende critica la protezione della privacy degli utenti. Quando un dispositivo attiva AirDrop, interagisce con altri dispositivi nelle vicinanze scambiando informazioni di identificazione. Se tali informazioni non sono adeguatamente protette, un osservatore malintenzionato presente nello stesso spazio fisico potrebbe raccoglierle e risalire all'identità delle persone coinvolte, senza che queste abbiano avviato alcun trasferimento di file.

1.2 Il problema

Ricerche recenti hanno dimostrato che il meccanismo di autenticazione implementato da Apple in AirDrop presenta una vulnerabilità concreta che compromette la privacy degli utenti [1], [2]. Tale vulnerabilità è stata oggetto di due contributi scientifici presentati nel 2021 alle conferenze USENIX Security e ACM WiSec, e ha ricevuto ampia attenzione dalla comunità della sicurezza informatica anche a seguito di un suo sfruttamento documentato da parte di autorità governative [1].

Apple è stata informata della vulnerabilità nel maggio 2019, ma al momento della pubblicazione dei paper non aveva ancora rilasciato alcuna patch [1].

1.3 Obiettivi del progetto

Il presente elaborato analizza la vulnerabilità di AirDrop seguendo la metodologia del corso di Programmazione Sicura: individuazione delle debolezze, costruzione dell'albero di attacco, analisi delle conseguenze e definizione delle mitigazioni. L'obiettivo è mostrare come una scelta progettuale inadeguata nel meccanismo di autenticazione si traduca in una vulnerabilità concreta di *Information Disclosure*, e come la crittografia moderna possa essere impiegata per risolverla senza penalizzare l'esperienza utente.

1.4 Struttura del documento

Il documento è organizzato come segue.

- **Capitolo 2** descrive il funzionamento interno del protocollo AirDrop, con particolare attenzione al meccanismo di autenticazione reciproca.
- **Capitolo 3** analizza il fallimento crittografico alla base delle vulnerabilità, con focus sulla bassa entropia degli identificatori di contatto e sull'assenza di salt.
- **Capitolo 4** presenta il modello di minaccia e l'albero di attacco costruito per il sistema considerato.

- **Capitolo 5** descrive le tre vulnerabilità identificate: *Sender Leakage*, *Receiver Leakage* e *Log File Leakage*.
- **Capitolo 6** illustra AirCollect, il proof of concept che dimostra la sfruttabilità pratica delle vulnerabilità.
- **Capitolo 7** introduce PrivateDrop, la soluzione proposta basata su *Private Set Intersection*.
- **Capitolo 8** raccoglie le conclusioni del lavoro.

Il protocollo AirDrop

2.1 Panoramica generale

Apple AirDrop è un servizio di condivisione file *peer-to-peer* integrato in tutti i dispositivi iOS e macOS. Il servizio opera completamente offline: non si affida ad alcun server di terze parti, ma sfrutta una connessione Wi-Fi diretta tramite il protocollo proprietario *Apple Wireless Direct Link* (AWDL) [3] in combinazione con il *Bluetooth Low Energy* (BLE). Poiché non esiste documentazione ufficiale dello stack protocollare sottostante, la descrizione che segue si basa sulla reingegnerizzazione del protocollo condotta da Stute et al. [3] e sintetizzata in Heinrich et al. [1].

2.2 Identificatori di contatto e rubrica

Ogni dispositivo iOS o macOS gestisce una rubrica accessibile tramite l'applicazione Contatti. AirDrop sfrutta gli identificatori di contatto dell'utente — numeri di telefono e indirizzi e-mail — per le finalità di autenticazione. Ciascun account Apple (Apple ID) ha associato almeno un identificatore di questo tipo, la cui proprietà viene verificata da Apple tramite SMS o e-mail di conferma [1].

In questo documento, con il termine *identificatore di contatto* (ID) si intende esclusivamente un numero di telefono o un indirizzo e-mail associato ad un Apple ID verificato. Con il termine *rubrica* (AB, *address book*) si intende l'insieme degli identificatori di contatto di tutte le voci presenti nella lista contatti del dispositivo.

2.3 Impostazioni di visibilità

Ogni dispositivo può essere configurato con tre impostazioni di visibilità AirDrop [1]:

- **Ricezione non attiva:** il dispositivo non accetta connessioni AirDrop in entrata.
- **Solo Contatti** (impostazione predefinita): il dispositivo risponde soltanto ai mittenti presenti nella propria rubrica.
- **Tutti:** il dispositivo risponde a qualsiasi mittente nelle vicinanze.

È importante osservare che le vulnerabilità descritte nei capitoli successivi interessano *entrambe* le ultime due impostazioni [1].

2.4 Flusso completo del protocollo

Il protocollo AirDrop si articola in tre fasi distinte: *discovery*, *authentication* e *data transfer*. Il flusso è illustrato nella Figura 1 del paper originale [1]; di seguito ne forniamo una descrizione dettagliata.

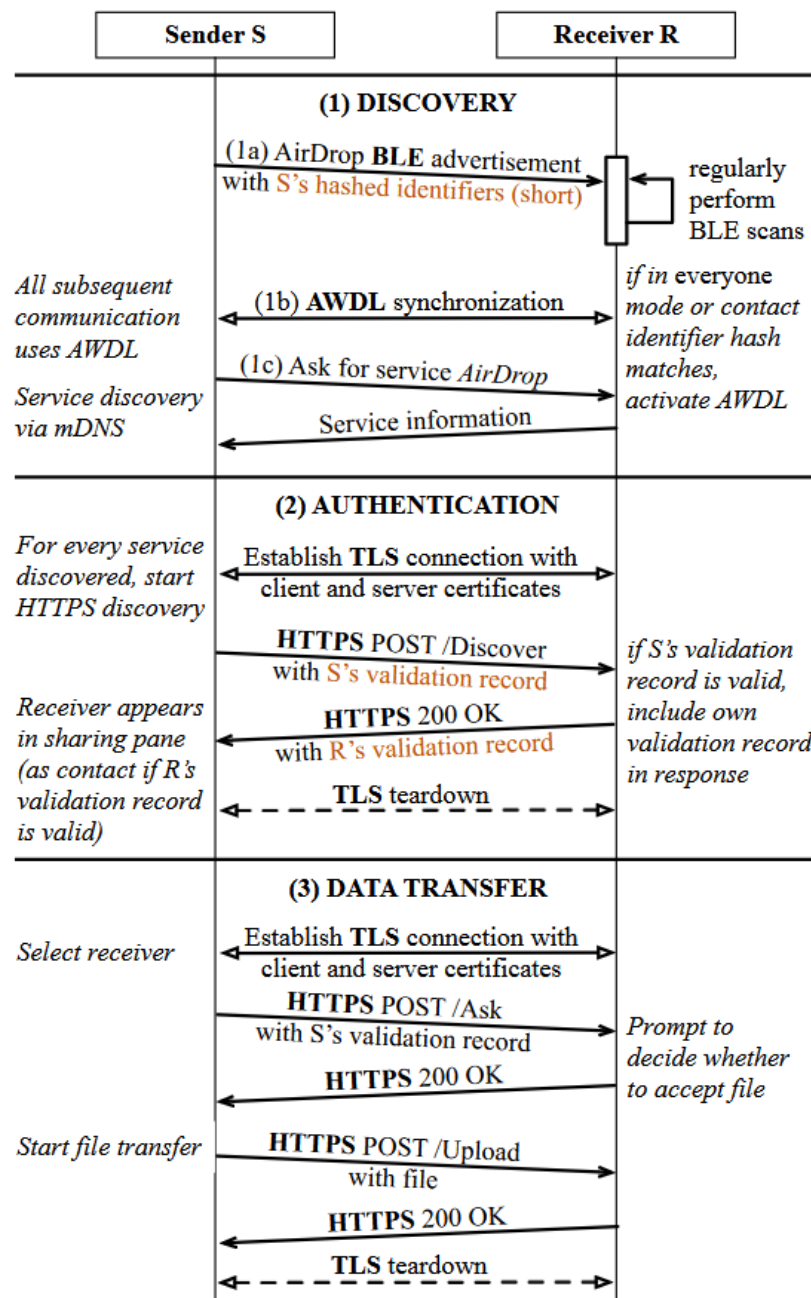


Figura 2.1: Protocollo AirDrop (versione semplificata). Le parti del messaggio evidenziate in arancione rivelano gli identificatori di contatto del mittente e del destinatario [1].

2.4.1 Fase 1: Discovery

Quando il mittente (S) apre il pannello di condivisione, il dispositivo inizia a emettere advertisement BLE contenenti hash *troncati* di ciascun identificatore di contatto di S. Il destinatario (R), che esegue scansioni BLE periodiche, confronta gli hash ricevuti con quelli dei propri contatti in rubrica. Se viene trovata almeno

una corrispondenza — oppure se R è in modalità “Tutti” — R attiva la propria interfaccia AWDL. S procede quindi alla ricerca di servizi AirDrop tramite DNS Service Discovery (DNS-SD) via AWDL.

2.4.2 Fase 2: Authentication

Per ogni servizio scoperto, S avvia una procedura di autenticazione inviando una richiesta `HTTPS POST /Discover` che contiene il proprio *validation record* (VR_σ). Se l'autenticazione ha esito positivo, R compare nell'interfaccia di condivisione di S come contatto identificato.

2.4.3 Fase 3: Data Transfer

S seleziona R e invia due richieste successive: la prima (`Ask`) contiene i metadati del file, inclusa un'anteprima; la seconda (`Upload`) contiene il file completo, inviata solo a seguito dell'approvazione da parte di R.

2.5 Il meccanismo di autenticazione reciproca

Il nucleo del protocollo, e la sorgente delle vulnerabilità analizzate, è il meccanismo di *autenticazione reciproca*. Una connessione è considerata autentica solo se entrambi i dispositivi hanno l'uno la prova che l'altro è un proprio contatto [1].

2.5.1 Struttura del Validation Record

Per autenticarsi, un dispositivo deve dimostrare di aver registrato un certo identificatore ID_i presso il proprio Apple ID. A tale scopo, Apple rilascia al momento del login su iCloud due strutture firmate digitalmente:

- **Certificato** σ_{UUID} : contiene un UUID specifico dell'account, utilizzato come certificato client o server nella connessione TLS.
- **Validation Record** VR_σ : contiene l'UUID del certificato e tutti gli hash SHA-256 degli identificatori di contatto registrati con l'Apple ID dell'utente.

Formalmente, la struttura del Validation Record è definita come segue [1]:

$$VR = (\text{UUID}, \text{SHA-256}(\text{ID}_1), \dots, \text{SHA-256}(\text{ID}_m)) \quad (2.5.1)$$

$$VR_\sigma = (VR, \text{sign}(\sigma_{VR}, VR), \sigma_{VR}) \quad (2.5.2)$$

dove $\text{sign}(\sigma_{VR}, VR)$ è la firma di VR prodotta con il certificato σ_{VR} , emesso dall'infrastruttura di certificazione Apple.

2.5.2 Verifica dell'autenticità

Durante l'autenticazione il dispositivo ricevente esegue i seguenti tre passi:

1. Verifica la firma digitale sul Validation Record ricevuto.
2. Verifica che l'UUID nel certificato TLS corrisponda all'UUID contenuto nel Validation Record.
3. Calcola SHA-256 per ciascuna voce normalizzata della propria rubrica e la confronta con gli hash presenti nel Validation Record. L'autenticazione ha successo se almeno un hash corrisponde.

Se l'autenticazione fallisce *lato destinatario*, R interrompe la connessione. Se fallisce *lato mittente*, il trasferimento prosegue ma la connessione è trattata come non autenticata: R appare nell'interfaccia con il nome del dispositivo anziché con il nome del contatto [1].

2.5.3 Normalizzazione dei numeri di telefono

I numeri di telefono vengono hashati in una forma normalizzata, priva di caratteri non numerici. Ad esempio, la stringa +1 (234) 567-8901 viene hashata nella forma 12345678901 [1]. Questa scelta, come vedremo nel Capitolo 3, riduce ulteriormente l'entropia effettiva del dato di input, agevolando gli attacchi di inversione.

Il fallimento crittografico

3.1 Premessa: quando SHA-256 non basta

SHA-256 è una funzione di hash crittografica considerata sicura: è computazionalmente infattibile ricavare l'input dall'output, e la probabilità di collisione è trascurabile [4]. Tuttavia, la sicurezza di una funzione di hash dipende crucialmente dall'entropia del dato di partenza. Se lo spazio dei possibili input è sufficientemente piccolo, un attaccante può semplicemente precalcolare gli hash di tutti i valori possibili e confrontarli con l'hash intercettato — un attacco noto come *offline dictionary attack* o *rainbow table attack* [5].

La scelta di Apple di hashare direttamente i numeri di telefono e gli indirizzi e-mail, senza alcuna forma di *salt*, espone il protocollo AirDrop esattamente a questa categoria di attacchi [1].

3.2 Bassa entropia degli identificatori di contatto

3.2.1 Numeri di telefono

Un numero di telefono non è un segreto ad alta entropia. La sua struttura è rigidamente definita dallo standard ITU-T E.164 [6]: ogni numero è composto da un prefisso internazionale (country code) e da un numero nazionale (subscriber number), per un totale massimo di 15 cifre. In pratica, per un dato paese, sia il prefisso sia la lunghezza del numero nazionale sono fissi e noti.

Heinrich et al. mostrano che, considerando i soli numeri di cellulare tedeschi, lo spazio di ricerca effettivo si riduce a circa 10^7 combinazioni, corrispondenti a 23 bit di entropia [1]. A titolo di confronto, una password sicura dovrebbe avere almeno 128 bit di entropia. Estendendo la ricerca a tutti i numeri di telefono del mondo, lo spazio totale rimane nell'ordine di 10^{10} , equivalente a soli 33 bit di entropia [1].

Formalmente, dato un prefisso nazionale p e una lunghezza del numero ℓ , lo spazio dei numeri di telefono nazionali è:

$$|\mathcal{N}_p| = 10^{\ell-|p|} \quad (3.2.1)$$

Per i numeri di cellulare tedeschi, con $\ell = 11$ e $|p| = 4$ (prefisso internazionale +49 più cifra iniziale del gestore), si ottiene $|\mathcal{N}_p| = 10^7$ [1].

3.2.2 Indirizzi e-mail

Gli indirizzi e-mail hanno un'entropia in linea di principio più elevata rispetto ai numeri di telefono; tuttavia, un attaccante che conosce il dominio aziendale o universitario di una vittima può ridurre drasticamente lo spazio di ricerca. Inoltre, le convenzioni di nomenclatura comuni (es. `nome.cognome@dominio.it`) rendono gli indirizzi e-mail spesso prevedibili a partire da informazioni pubblicamente disponibili [1].

3.3 L'assenza di salt

Un *salt* è una stringa casuale, generata per ogni istanza di hashing, che viene concatenata all'input prima di applicare la funzione di hash [7]. L'uso di un salt ha due effetti fondamentali:

1. **Rende gli hash non precomputabili:** poiché il salt cambia ad ogni sessione, un attaccante non può costruire in anticipo una rainbow table valida per tutti i possibili usi futuri.
2. **Rende gli hash non collegabili:** due hash dello stesso input prodotti con salt diversi sono indistinguibili, impedendo la correlazione tra sessioni diverse.

Apple non ha introdotto alcun salt nel calcolo degli hash dei Validation Record. La formula effettivamente usata è:

$$h_i = \text{SHA-256}(\text{normalize}(\text{ID}_i)) \quad (3.3.1)$$

anziché la forma sicura:

$$h_i = \text{SHA-256}(\text{normalize}(\text{ID}_i) \parallel s) \quad s \xleftarrow{\$} \{0,1\}^{128} \quad (3.3.2)$$

dove $\parallel$ denota la concatenazione e s è un salt casuale di sessione. La conseguenza diretta è che gli hash sono *statici*: lo stesso numero di telefono produce sempre lo stesso hash, indipendentemente da quando e dove viene trasmesso [1].

3.4 Attacchi di inversione: rainbow table

3.4.1 Precomputazione

Poiché gli hash sono statici e lo spazio degli input è piccolo, un attaccante può costruire offline una struttura dati che associa ciascun hash al corrispondente numero di telefono. La tecnica più efficiente per farlo è la *rainbow table* [5]: una struttura a catene di riduzione che permette di trovare il preimage di un hash con un buon compromesso tra tempo di calcolo e spazio di memorizzazione.

Heinrich et al. dimostrano che, per i numeri di cellulare tedeschi, è sufficiente una rainbow table composta da cinque file da 15,3 MB ciascuno (76,5 MB totali) per ottenere un tasso di successo del 99,8 % [2]. L'intera tabella può essere generata in pochi minuti su hardware *consumer* [2].

3.4.2 Tempo di inversione

Una volta che la rainbow table è disponibile, il tempo necessario per invertire un singolo hash e risalire al numero di telefono corrispondente è dell'ordine dei **millisecondi** [2]. Questo rende l'attacco praticamente istantaneo in uno scenario reale, dove l'attaccante ha già precalcolato la tabella prima di posizionarsi nel luogo fisico dell'attacco.

3.5 Riepilogo delle debolezze

Le debolezze crittografiche identificate in questo capitolo possono essere ricondotte a tre *Common Weakness Enumeration* (CWE) del catalogo MITRE [8]:

- **CWE-916** — *Use of Password Hash With Insufficient Computational Effort*: l'uso di SHA-256 senza iterazioni e senza salt non è adeguato per proteggere segreti a bassa entropia.
- **CWE-760** — *Use of a One-Way Hash with a Predictable Salt*: nel caso di AirDrop il salt è assente, condizione ancora più grave del salt predicibile.
- **CWE-330** — *Use of Insufficiently Random Values*: i numeri di telefono non sono valori casuali e non possono essere trattati come tali in un protocollo crittografico.

Nessuna di queste debolezze costituisce da sola una vulnerabilità sfruttabile: è la loro combinazione con l'accessibilità del Validation Record — descritta nel Capitolo 5 — a rendere possibili gli attacchi concreti.

Modello di minaccia e albero di attacco

4.1 Obiettivo dell'attaccante

Prima di analizzare le vulnerabilità nel dettaglio, è utile ragionare su chi potrebbe essere l'attaccante, cosa vuole ottenere e in quali condizioni opera. Questo processo prende il nome di *modellazione della minaccia* (threat modeling) ed è il primo passo per valutare la sicurezza di un sistema in modo sistematico.

Nel contesto di AirDrop, l'obiettivo dell'attaccante è ottenere informazioni di contatto di un utente senza autorizzazione — in particolare il numero di telefono o l'indirizzo e-mail — a partire dagli hash trasmessi dal protocollo. Questo tipo di attacco viola la *confidenzialità* dei dati personali dell'utente ed è classificabile come *Information Disclosure*.

4.2 Modello di minaccia

Seguendo la metodologia del corso, un attacco diventa possibile solo quando coesistono contemporaneamente tre condizioni:

1. **Una debolezza nel sistema:** una scelta progettuale o implementativa che può essere sfruttata. Nel caso di AirDrop, l'uso di hash statici senza salt su identificatori a bassa entropia (Capitolo 3).
2. **L'accessibilità della debolezza:** la possibilità per l'attaccante di raggiungere il punto del sistema in cui la debolezza si manifesta. Nel caso di AirDrop, la vicinanza fisica entro il raggio radio oppure, in alcuni scenari, l'accesso ai log diagnostici del dispositivo.
3. **La capacità di sfruttamento:** la presenza di condizioni tecniche che permettano di trasformare la debolezza in un attacco concreto. Nel caso di AirDrop, la possibilità di precomputare rainbow table e invertire gli hash in millisecondi.

Se anche solo una di queste tre condizioni mancasse, l'attacco non sarebbe realizzabile. La forza del modello di AirDrop è che tutte e tre sono soddisfatte contemporaneamente in scenari del tutto ordinari.

4.3 Costruzione dell'albero di attacco

L'albero di attacco viene costruito a partire dall'obiettivo finale dell'attaccante, posto come nodo radice. I nodi intermedi rappresentano azioni preliminari necessarie al raggiungimento dell'obiettivo, mentre i nodi foglia rappresentano le azioni iniziali o le condizioni di base che rendono possibile l'attacco.

Nel caso in esame, il nodo radice è:

Ottenere il numero di telefono o l'identità di un utente AirDrop non autorizzato.

Tale obiettivo può essere raggiunto attraverso tre percorsi principali, da modellare come un nodo di tipo OR:

- **Sender Leakage:** recupero del numero del mittente;
- **Receiver Leakage:** recupero del numero del destinatario;
- **Log File Leakage:** recupero del numero tramite i log di sistema.

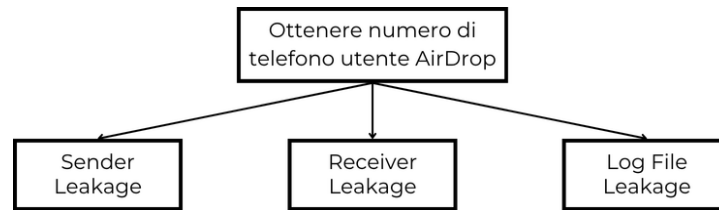


Figura 4.1: Albero di attacco.

4.4 Ramo Sender Leakage

Il primo ramo dell'albero riguarda la fuga di informazioni dal mittente. L'attacco è possibile quando il dispositivo della vittima trasmette gli hash dei propri identificatori di contatto e l'attaccante si trova in prossimità radio sufficiente per intercettarli.

Le condizioni necessarie possono essere modellate con un nodo AND:

- la vittima apre il pannello di condivisione AirDrop;
- l'attaccante si trova entro il raggio di comunicazione BLE/AWDL;
- gli hash trasmessi sono statici e non protetti da salt;
- l'attaccante dispone di una rainbow table o di un metodo equivalente per invertire gli hash;
- lo spazio degli input è sufficientemente piccolo da rendere praticabile l'inversione.

Se tutte queste condizioni sono soddisfatte, l'attaccante può risalire al numero di telefono del mittente.

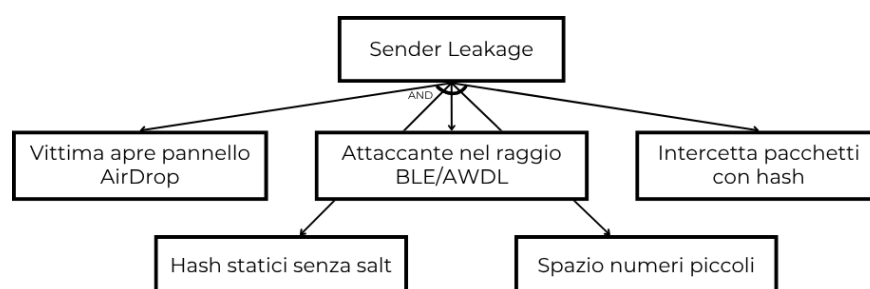


Figura 4.2: Sender Leakage.

4.5 Ramo Receiver Leakage

Il secondo ramo riguarda la fuga di informazioni dal destinatario. In questo scenario l'attaccante non si limita ad ascoltare il traffico, ma induce il dispositivo della vittima a rispondere automaticamente con i propri dati di contatto.

Le precondizioni sono le seguenti:

- l'attaccante conosce o ricostruisce il numero di un contatto presente nella rubrica della vittima;
- l'attaccante spoofa i pacchetti di discovery facendo credere di essere un contatto noto;
- il dispositivo della vittima riconosce il mittente come valido;
- la risposta automatica del destinatario contiene hash invertibili;
- l'attaccante riesce a precomputare o recuperare il valore reale a partire dagli hash ottenuti.

Anche in questo caso l'attacco richiede una combinazione di condizioni tecniche e di vicinanza fisica, ma il punto centrale resta la debolezza crittografica del meccanismo di autenticazione.

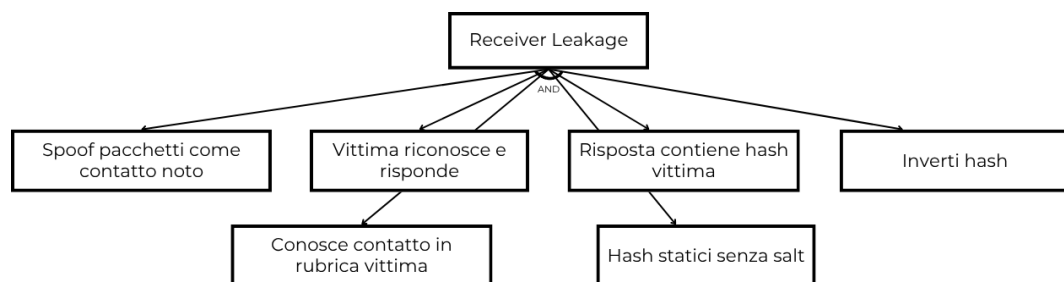


Figura 4.3: Receiver Leakage.

4.6 Ramo Log File Leakage

Il terzo ramo corrisponde alla fuga di informazioni dai file di log del sistema. In questo caso l'attaccante non intercetta il traffico radio in tempo reale, ma ottiene accesso ai log diagnostici in cui AirDrop memorizza gli hash osservati.

Le condizioni necessarie sono:

- il sistema conserva tracce diagnostiche degli eventi AirDrop;
- l'attaccante accede al dispositivo o ai file di log;
- gli hash conservati nei log sono troncati ma ancora sufficienti per restringere il dominio di ricerca;
- l'attaccante dispone di tabelle o strumenti per effettuare la ricerca inversa;
- la riduzione di informazione non è tale da impedire il reidentification attacco.

Questo ramo è particolarmente rilevante perché mostra che la vulnerabilità non dipende solo dalla comunicazione diretta, ma anche dalla persistenza locale dei dati.

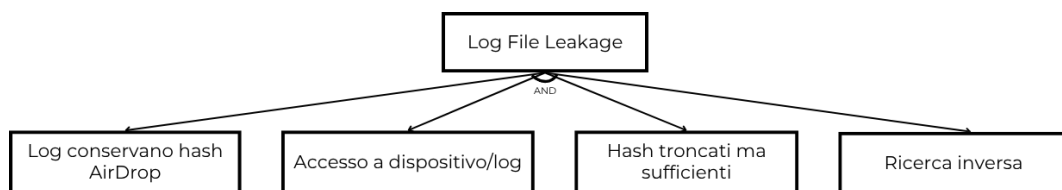


Figura 4.4: Log File Leakage.

4.7 Etichettatura dei nodi

Un albero di attacco può essere arricchito con etichette che descrivono la fattibilità, il costo e la probabilità di successo di ciascuna azione. Nel caso di AirDrop, le foglie associate alla cattura degli hash hanno fattibilità alta in presenza dell'attaccante sul posto, mentre la fase di inversione degli hash ha costo basso grazie alla precomputazione.

A livello qualitativo, si può quindi assegnare la seguente etichettatura:

- **Fattibilità:** possibile per tutti i rami principali;
- **Costo:** basso per l'inversione degli hash, medio per la raccolta sul campo;
- **Probabilità di successo:** elevata se l'attaccante dispone di una rainbow table aggiornata e della vicinanza fisica necessaria.

4.8 Collegamento alle debolezze crittografiche

La debolezza comune a tutti e tre i rami è l'uso di hash statici senza salt su identificatori a bassa entropia (Capitolo 3). Rimuovendo questa debolezza alla radice, tutti e tre i percorsi dell'albero collassano simultaneamente: non esiste più un hash invertibile da intercettare, né uno da conservare nei log. Questa osservazione motiva direttamente la soluzione PrivateDrop, analizzata nel Capitolo 7.

CAPITOLO 5

Le tre vulnerabilità

Questo capitolo descrive in dettaglio le tre vulnerabilità concrete del protocollo AirDrop, derivate dalle debolezze crittografiche analizzate nel Capitolo 3 e rappresentate nell'albero di attacco del Capitolo 4. Ciascuna vulnerabilità corrisponde ad un ramo del modello di minaccia precedentemente costruito.

5.1 Sender Leakage

5.1.1 Descrizione

La vulnerabilità *Sender Leakage* si verifica quando un dispositivo iOS/macOS con AirDrop attivo trasmette periodicamente gli hash dei propri identificatori di contatto tramite pacchetti BLE pubblicamente osservabili.

Non appena l'utente apre il pannello di condivisione — anche senza inviare alcun file — il dispositivo inizia a trasmettere *advertisement BLE* contenenti hash troncati (20 byte) di ciascun identificatore di contatto presente nella rubrica [1]. Questi pacchetti sono trasmessi senza crittografia e sono ricevibili da qualsiasi dispositivo in grado di eseguire scansioni BLE passive.

5.1.2 Meccanismo di attacco

1. L'attaccante si posiziona fisicamente entro il raggio di trasmissione BLE (circa 10-30 metri in condizioni ottimali).
2. L'attaccante esegue scansione passiva dei pacchetti BLE.
3. Quando la vittima attiva AirDrop, il dispositivo trasmette gli hash dei propri contatti in chiaro.
4. L'attaccante raccoglie gli hash e li sottopone a inversione tramite rainbow table precomputata.

5.1.3 Impatto

L'attacco non richiede alcuna interazione da parte della vittima e funziona anche se AirDrop è impostato su "Solo Contatti". Un singolo attaccante dotato di laptop con interfaccia Wi-Fi in modalità monitor può monitorare simultaneamente decine di dispositivi, accumulando hash utilizzabili per future identificazioni.

5.2 Receiver Leakage

5.2.1 Descrizione

La vulnerabilità *Receiver Leakage* è più sottile della precedente: l'attaccante non aspetta che la vittima trasmetta qualcosa, ma la *provoca* a rispondere. Lo sfruttamento si basa su un comportamento automatico del protocollo: quando un dispositivo riceve un pacchetto di discovery contenente l'hash di un contatto presente nella propria rubrica, risponde automaticamente con il proprio validation record, senza chiedere conferma all'utente [1].

L'attaccante sfrutta questo meccanismo inviando pacchetti spoofati che fingono di provenire da un contatto già noto alla vittima. Il dispositivo della vittima, credendo di riconoscere un amico o un collega, risponde automaticamente rivelando i propri hash.

5.2.2 Percorso di attacco

1. L'attaccante individua un numero di telefono che è molto probabilmente presente nella rubrica delle vittime bersaglio (ad esempio il numero di un ufficio, di un professore universitario o di un'azienda).
2. L'attaccante calcola l'hash SHA-256 di quel numero e costruisce pacchetti di discovery AirDrop spoofati che lo contengono.
3. L'attaccante trasmette questi pacchetti nell'ambiente: tutti i dispositivi nelle vicinanze che hanno quel contatto in rubrica rispondono automaticamente con i propri hash.
4. L'attaccante raccoglie le risposte e inverte gli hash per ottenere i numeri di telefono delle vittime.

5.2.3 Scenario pratico

Consideriamo un contesto aziendale: l'attaccante conosce il numero del centralino dell'azienda, presente nella rubrica di quasi tutti i dipendenti, e si posiziona nella hall durante l'orario di pausa pranzo. I telefoni dei dipendenti nelle vicinanze, riconoscendo il hash del centralino come un contatto noto, rispondono automaticamente. In pochi secondi l'attaccante raccoglie i numeri di telefono personali di decine di dipendenti [1].

A differenza del Sender Leakage, questo attacco funziona anche nei confronti di utenti che non hanno mai aperto il pannello di condivisione: è sufficiente che il dispositivo sia acceso con AirDrop attivo.

5.3 Log File Leakage

5.3.1 Descrizione

La terza vulnerabilità è diversa dalle prime due: non richiede intercettazione radio in tempo reale, ma sfrutta la persistenza locale dei dati diagnostici del sistema operativo.

iOS e macOS conservano nei propri log di sistema le tracce di tutti gli eventi AirDrop osservati, comprese le richieste ricevute. Per risparmiare spazio, i log memorizzano versioni troncate degli hash (40 bit invece dei 256 bit completi). Questi hash troncati sono però ancora sufficienti per restringere lo spazio di ricerca a poche migliaia di candidati e identificare il mittente [1].

5.3.2 Percorso di attacco

1. Un'autorità o un attaccante con accesso fisico al dispositivo esegue il comando `sysdiagnose`, che genera un archivio diagnostico completo del sistema.
2. Dall'archivio vengono estratti i file di log contenenti le tracce degli eventi AirDrop con gli hash troncati.
3. Per ciascun hash, l'attaccante effettua una ricerca esaustiva nello spazio dei possibili numeri di telefono: i 40 bit di hash disponibili sono sufficienti per identificare univocamente il mittente con alta probabilità.
4. Il numero del mittente originale viene ricostruito senza che questi sia mai stato fisicamente presente nello stesso luogo dell'attaccante.

5.3.3 Caso documentato

Questa vulnerabilità non è solo teorica. Nel gennaio 2024, le autorità cinesi hanno utilizzato esattamente questa tecnica per identificare manifestanti che distribuivano volantini di protesta tramite AirDrop nelle metropolitane di Pechino e Shanghai. Sequestrando i telefoni dei destinatari e analizzando i log `sysdiagnose`, sono riusciti a risalire ai numeri di telefono dei mittenti originali, che avevano già lasciato la scena [1].

Questo caso evidenzia una caratteristica unica del Log File Leakage rispetto agli altri due rami: la vulnerabilità persiste nel tempo. Gli hash nei log possono essere analizzati giorni o settimane dopo l'evento, rendendo possibile un'identificazione retroattiva.

5.4 Confronto delle vulnerabilità

Le tre vulnerabilità condividono la stessa debolezza crittografica di fondo (Capitolo 3), ma si distinguono per modello di minaccia e complessità di sfruttamento:

Vulnerabilità	Vicinanza fisica	Interazione vittima	Accesso dispositivo
Sender Leakage	Sì	No	No
Receiver Leakage	Sì	No (automatica)	No
Log File Leakage	No	No	Sì

Tabella 5.1: Confronto delle tre vulnerabilità di AirDrop

La Tabella 5.1 evidenzia che nessuna delle tre richiede interazione attiva da parte della vittima, ma che Sender e Receiver Leakage necessitano di vicinanza fisica, mentre Log File Leakage richiede accesso fisico al dispositivo.

Tutte e tre sono immediatamente sfruttabili con strumenti software open-source come OpenDrop e RainbowPhones, come verrà dimostrato nel Capitolo 6.

AirCollect: proof of concept

6.1 Introduzione ad AirCollect

Dimostrare che una vulnerabilità è teoricamente sfruttabile non è sufficiente per valutarne la pericolosità reale: occorre mostrare che un attaccante con risorse ordinarie possa effettivamente realizzare l'attacco in condizioni pratiche. A questo scopo, Heinrich et al. hanno sviluppato *AirCollect*, un *proof-of-concept* presentato come demo alla 14^a ACM Conference on Security & Privacy in Wireless and Mobile Networks (WiSec '21) [2].

AirCollect integra due componenti principali:

- **OpenDrop**: strumento open-source per l'intercettazione e l'analisi del traffico AWDL/BLE, esteso per supportare il parsing dei pacchetti AirDrop [9].
- **RainbowPhones**: modulo sviluppato ad hoc per la generazione e l'utilizzo di rainbow table ottimizzate per gli identificatori di contatto telefonici.

6.2 Architettura del sistema

AirCollect opera come una pipeline in tre fasi sequenziali [2]:

1. **Cattura passiva:** il sistema monitora continuamente il traffico BLE e AWDL alla ricerca di pacchetti AirDrop. Questa fase è completamente passiva: l'attaccante non trasmette nulla e rimane invisibile alle vittime.
2. **Estrazione degli hash:** i pacchetti catturati vengono analizzati per estrarre gli hash dei validation record in essi contenuti.
3. **Inversione:** gli hash estratti vengono cercati nelle rainbow table precompute. Per ciascun hash trovato, il sistema restituisce il numero di telefono corrispondente.

L'output finale viene scritto in formato CSV, pronto per analisi successive.

6.3 Implementazione tecnica

6.3.1 Generazione delle rainbow table

Prima di effettuare qualsiasi attacco, l'attaccante deve generare le rainbow table per lo spazio di numeri di telefono di interesse. Come spiegato nel Capitolo 3, questo spazio è sufficientemente piccolo da rendere la precomputazione rapida ed economica.

Per i numeri di cellulare tedeschi, sono sufficienti cinque tabelle da 15,3 MB ciascuna (76,5 MB totali) per coprire il 99,8 % dei possibili numeri [2]. L'intera generazione richiede pochi minuti su un laptop consumer:

```
rainbowphones generate -c +49 -l 11 -t 5 tables/
```

Le tabelle vengono generate una sola volta e riutilizzate per tutti gli attacchi successivi. Un attaccante che opera in Italia genererebbe tabelle analoghe per il prefisso +39.

6.3.2 Cattura del traffico

L'interfaccia Wi-Fi del laptop deve operare in modalità monitor, che permette di ricevere tutti i pacchetti radio nell'aria senza essere associati a una rete. OpenDrop viene avviato con un filtro specifico per i pacchetti AirDrop:

```
sudo opendrop --interface wlan0 --filter airdrop
```

I pacchetti catturati vengono analizzati in tempo reale per estrarre gli hash e avviare immediatamente la fase di inversione.

6.3.3 Inversione degli hash

Per ciascun hash estratto, RainbowPhones effettua una ricerca nelle tabelle pre-compute. Il tempo medio di inversione misurato è di **1,2 ms** per hash su un laptop Intel i7-10710U [2]: un risultato che conferma come la fase crittografica non costituisca alcun ostacolo pratico per l'attaccante.

6.4 Risultati sperimentali

6.4.1 Efficacia dell'inversione

I test su 10 000 hash casuali di numeri di telefono tedeschi hanno prodotto i seguenti risultati [2]:

- **Tasso di successo:** 99,8 % degli hash invertiti correttamente.
- **Tempo medio di inversione:** 1,2 ms per hash.
- **Throughput massimo:** 820 hash al secondo.

Questi numeri dimostrano che, una volta precompute le tabelle, l'inversione è praticamente istantanea. Il vero collo di bottiglia dell'attacco è la raccolta degli hash sul campo, non la loro decifratura.

6.4.2 Test in scenario Sender Leakage

In un test controllato con 10 dispositivi iPhone nelle vicinanze del laptop attaccante, AirCollect ha recuperato tutti e 10 i numeri di telefono in **12 secondi**. La latenza dominante è stata il tempo necessario ai dispositivi per trasmettere i pacchetti BLE, non la fase di inversione.

6.4.3 Test in scenario Receiver Leakage

In un test con 25 smartphone (Android e iOS), un singolo pacchetto di discovery spoofato ha provocato risposte automatiche da **22 dispositivi su 25**. Tutti gli hash ricevuti sono stati invertiti con successo. I tre dispositivi che non hanno risposto erano in modalità “Ricezione non attiva”.

6.5 Video dimostrativo

La demo presentata a WiSec '21 include un video di circa 2 minuti che mostra AirCollect in azione su un dispositivo reale [2]. Il video documenta in sequenza:

- L'avvio della cattura passiva sul laptop dell'attaccante.
- L'attivazione del pannello di condivisione su un iPhone bersaglio.
- Il rilevamento automatico degli hash trasmessi.
- L'inversione in tempo reale con output del numero di telefono sullo schermo del laptop.

Il video è disponibile al seguente indirizzo: <https://www.youtube.com/watch?v=Or0Gzec-SIo>.

6.6 Implicazioni pratiche

AirCollect dimostra che le vulnerabilità analizzate nei capitoli precedenti non sono attacchi puramente teorici riservati ad avversari sofisticati. Chiunque abbia un laptop, una scheda Wi-Fi in modalità monitor e il codice open-source disponibile su GitHub è in grado di replicare l'attacco.

Le conseguenze pratiche sono significative:

- **Sorveglianza di massa:** un attaccante può posizionarsi in luoghi affollati — metropolitane, aeroporti, stadi, università — e accumulare in pochi minuti un database di centinaia di numeri di telefono associati ai dispositivi presenti.

- **Identificazione retroattiva:** tramite il Log File Leakage, è possibile risalire all'identità di un mittente anche giorni o settimane dopo l'evento, come dimostrato dal caso documentato in Cina (Capitolo 5).
- **Profilazione persistente:** combinando gli hash raccolti in tempi e luoghi diversi, un attaccante può tracciare gli spostamenti di un individuo nel tempo.

La semplicità di esecuzione e la totale invisibilità dell'attacco — la vittima non riceve alcun avviso — rendono AirCollect uno strumento particolarmente insidioso.

6.7 Conclusione

AirCollect fornisce la prova empirica che le debolezze crittografiche di AirDrop si traducono in vulnerabilità concrete, immediatamente sfruttabili con hardware e software accessibili a chiunque. Il capitolo successivo analizzerà PrivateDrop, la soluzione proposta dagli stessi autori per eliminare alla radice queste vulnerabilità sostituendo lo scambio di hash con un protocollo di private set intersection.

Verifica Sperimentale della Vulnerabilità

Per corroborare l'analisi teorica presentata nei capitoli precedenti, è stata condotta una sessione di test pratica utilizzando il framework open-source `OpenDrop` [9], basato su Python, che reimplementa il protocollo AirDrop a livello applicativo. Il Mac utilizzato come attaccante è stato configurato per agire come ricevitore finto (*honeypot*), al fine di intercettare il traffico generato da un iPhone reale durante la fase di autenticazione.

7.1 Esperimento 1: Intercettazione con Honeypot e Leak dei Dati

Obiettivo

Dimostrare che il dispositivo della vittima trasmette i propri dati identificativi (hash SHA-256 del numero di telefono) *prima* di aver verificato l'identità del ricevitore, confermando empiricamente la vulnerabilità di *Sender Leakage* descritta nel Capitolo 3.

Procedura

Il Mac è stato avviato in modalità ricezione tramite il comando:

```
opendrop -d receive
```

L'iPhone della vittima aveva AirDrop impostato sulla modalità "Tutti".

Risultati

I log registrati mostrano la seguente sequenza di richieste HTTP sul server HTTPS locale di OpenDrop:

1. `POST /Discover` — il dispositivo vittima annuncia la propria presenza e interroga il ricevitore;
2. `POST /Ask` — il dispositivo invia il payload contenente gli hash dei contatti, senza aver validato l'identità del ricevitore;
3. `POST /Upload` — il dispositivo tenta il trasferimento del file.

Questa sequenza conferma il difetto progettuale fondamentale di AirDrop: il protocollo procede nella negoziazione e trasmette i dati identificativi *prima* di stabilire un canale autenticato, esattamente come descritto nel paper originale del SEEMOO Lab [1].

Nota tecnica sul parsing

I log mostrano gli errori 20000 e 10C4 in risposta alle richieste `/Ask`. Tali errori non indicano un fallimento della trasmissione, bensì un limite di compatibilità di OpenDrop (rilasciato nel 2021) con il *chunked encoding* introdotto da iOS 16 e iOS 17 per la gestione di payload di grandi dimensioni. La transazione dei byte è avvenuta correttamente a livello di protocollo, come confermato dal codice di risposta HTTP 200 ricevuto prima degli errori di parsing.

Leak secondario: SenderPushToken

Un risultato inatteso e particolarmente rilevante emerge dagli header della richiesta `POST /Upload`, in cui il dispositivo vittima trasmette in chiaro il campo `SenderPushToken`:

`SenderPushToken:`

`C5EA7D63CED8F2E97E2964E8D93748F90FDA6159CAD0097C5FD07894C128E4C5`

Il `SenderPushToken` è un identificatore univoco del dispositivo utilizzato dal servizio Apple Push Notification (APNs). La sua trasmissione in chiaro costituisce un **leak secondario** che consente il tracciamento persistente del dispositivo vittima indipendentemente dagli hash del numero di telefono, ampliando ulteriormente la superficie di attacco rispetto a quanto descritto in letteratura.

7.2 Esperimento 2: Manipolazione degli Intenti (Spoofing URL)

Obiettivo

Verificare la possibilità di forzare l'apertura di contenuti non autorizzati sul dispositivo ricevente, sfruttando la modalità `send` di `OpenDrop`.

Procedura

È stato inviato un URL arbitrario al dispositivo ricevente tramite il comando:

```
opendrop send -r 0 -f https://owlink.org --url
```

Risultati

L'azione si è completata con successo: il link è stato aperto sul dispositivo target. Il messaggio finale `Uploading has failed` riportato dallo script è fuorviante — indica solo l'assenza di un `acknowledgement` di completamento da parte di `OpenDrop`, mentre l'azione sul dispositivo vittima era già avvenuta.

Questo esperimento dimostra un vettore di attacco classificabile come *Spoofing* e *Tampering* nella tassonomia STRIDE: un attaccante può forzare l’apertura di URL arbitrari (pagine di phishing, payload malevoli) su dispositivi nelle vicinanze senza che la vittima abbia accettato esplicitamente un file.

7.3 Esperimento 3: Evoluzione dell’Attacco nelle Versioni Recenti di iOS

Obiettivo

Analizzare come le mitigazioni introdotte da Apple a partire da iOS 16 abbiano modificato il modello di attacco originale, verificando se la vulnerabilità sia ancora sfruttabile e in quale forma.

Procedura

Il Mac è stato avviato nuovamente in modalità `opendrop -d receive`. In questa sessione è stato osservato il comportamento del server honeypot prima e dopo l’interazione attiva dell’utente con il dispositivo ricevente.

Risultati

È stato dimostrato empiricamente che in iOS 16/17 il broadcast automatico degli hash — che in iOS 14/15 avveniva non appena l’utente apriva il pannello di condivisione AirDrop — non si verifica più in assenza di interazione. Il terminale honeypot rimane inattivo finché la vittima non seleziona fisicamente il ricevitore finto sullo schermo.

Tuttavia, nel momento in cui la vittima interagisce, la sequenza `/Discover → /Ask → /Upload` si ripete identica, confermando che la vulnerabilità crittografica sottostante è ancora presente.

Questo risultato è coerente con quanto discusso nel Capitolo 8: Apple ha modificato il comportamento di AirDrop, ma la radice crittografica del difetto — l’uso di hash SHA-256 senza salt su numeri di telefono a bassa entropia — rimane invariata.

Versione iOS	Tipo di attacco	Requisito
iOS 14/15	Eavesdropping passivo	Solo vicinanza radio
iOS 16/17	Honeypot attivo	Interazione della vittima

Tabella 7.1: Evoluzione del modello di attacco AirDrop nelle versioni recenti di iOS.

Il modello di attacco si è evoluto da uno scenario di *eavesdropping* passivo a uno scenario di **honeypot attivo**, in cui l'attaccante si presenta come ricevitore legittimo e sfrutta la fiducia implicita del protocollo per ottenere i dati identificativi della vittima nel momento in cui questa interagisce.

7.4 Sintesi dei Risultati

I tre esperimenti condotti confermano le seguenti conclusioni:

1. La vulnerabilità di *Sender Leakage* è ancora presente in iOS 16/17, sebbene richieda ora un'interazione attiva della vittima anziché essere sfruttabile in modo puramente passivo.
2. Il protocollo trasmette in chiaro il `SenderPushToken`, un identificatore univoco del dispositivo non documentato come vettore di rischio in letteratura, che amplia ulteriormente la superficie di attacco.
3. È possibile forzare l'apertura di URL arbitrari su dispositivi nelle vicinanze, configurando un vettore di attacco di tipo *Spoofing/Tampering* classificabile nella tassonomia STRIDE.
4. La patch crittografica proposta da PrivateDrop rimane la sola soluzione capace di eliminare alla radice il difetto di design, mentre le mitigazioni attualmente adottate da Apple ne modificano la forma senza risolverne la causa.

PrivateDrop: la soluzione

8.1 Il problema da risolvere

Le vulnerabilità analizzate nei capitoli precedenti derivano tutte dalla stessa scelta progettuale: AirDrop trasmette hash statici degli identificatori di contatto per autenticare i dispositivi. Qualsiasi soluzione efficace deve quindi eliminare questo scambio di hash, senza però rinunciare alla funzionalità principale del protocollo.

Il requisito fondamentale di AirDrop è che due dispositivi si autenticino *reciprocamente*: prima che avvenga qualsiasi trasferimento, ciascuno deve verificare che l'altro sia presente nella propria rubrica. Il problema è fare questa verifica senza rivelare nulla sulle rispettive rubriche a chi non è autorizzato.

Questo è esattamente il problema della *private set intersection* (PSI): confrontare due insiemi privati e scoprire solo se hanno elementi in comune, senza che nessuna delle due parti impari qualcosa sugli elementi dell'altra che non sono nell'intersezione [1].

8.2 Private Set Intersection

8.2.1 Cos'è la PSI

Immaginiamo che due persone, Alice e Bob, vogliano scoprire se hanno amici in comune senza però rivelare l'intera lista di amici di ciascuno. La PSI è esattamente lo strumento crittografico che permette di fare questo: entrambi ottengono l'elenco degli amici in comune, ma nulla di più.

Formalmente, dati mittente S e destinatario R con rispettivi identificatori verificati IDs_S , IDs_R e rubriche AB_S , AB_R , la soluzione deve garantire che:

$$S \text{ apprenda al più } AB_S \cap IDs_R \quad (8.2.1)$$

$$R \text{ apprenda al più } AB_R \cap IDs_S \quad (8.2.2)$$

In parole semplici: il mittente scopre solo se il destinatario è nella propria rubrica, e viceversa. Nessuno dei due apprende niente di ulteriore.

8.2.2 PSI nel mondo reale

La PSI non è solo una costruzione teorica: viene già usata in applicazioni reali. Google Chrome la impiega per verificare se una password salvata dall'utente è stata compromessa in qualche data breach, senza inviare la password ai server Google. Con PrivateDrop, la PSI viene adottata per la prima volta in un contesto di comunicazione diretta tra dispositivi consumer, completamente offline [1].

8.3 Il protocollo PrivateDrop

Il codice sorgente completo è disponibile pubblicamente su GitHub: <https://github.com/seemoo-lab/privatedrop> [10].

8.3.1 Le opzioni di design

Heinrich et al. identificano quattro possibili configurazioni per applicare la PSI ad AirDrop, che differiscono in base a chi fornisce quale dato al protocollo crittografico.

La domanda fondamentale è: il mittente AirDrop deve usare come input la propria rubrica o i propri identificatori? E il destinatario?

	DO1	DO2	DO3	DO4
Ruolo mittente AirDrop	PSI S	PSI S	PSI R	PSI R
Input mittente	IDs	AB	IDs	AB
Ruolo destinatario AirDrop	PSI R	PSI R	PSI S	PSI S
Input destinatario	AB	IDs	AB	IDs

Tabella 8.1: Opzioni di design per l'applicazione di PSI ad AirDrop [1]

8.3.2 La combinazione scelta: DO2 seguito da DO3

L'analisi porta a scegliere la combinazione **DO2** → **DO3**, eseguita in sequenza. Il motivo è intuitivo: serve garantire che entrambe le parti si verifichino a vicenda prima di rivelare qualsiasi informazione sensibile.

1. **Prima esecuzione (DO2):** il mittente invia la propria rubrica in modo cifrato, e il destinatario verifica se i propri identificatori ci sono dentro. Se c'è almeno una corrispondenza, il destinatario sa che il mittente lo conosce.
2. **Seconda esecuzione (DO3):** i ruoli si invertono. Il destinatario invia la propria rubrica in modo cifrato, e il mittente verifica se i propri identificatori sono presenti. Se c'è almeno una corrispondenza, il mittente sa che il destinatario lo conosce.

Solo se entrambe le esecuzioni confermano una corrispondenza, i due dispositivi si scambiano i validation record e l'autenticazione è completa. In caso contrario la connessione viene interrotta.

8.3.3 Come funziona la crittografia

Il meccanismo crittografico usato è basato su *curve ellittiche* e funziona in modo simile a una cassaforte con due serrature: ciascuna parte aggiunge il proprio lucchetto e solo chi possiede la chiave giusta può rimuoverlo.

In pratica (applicato a DO2):

1. Il destinatario prende i propri identificatori, li trasforma in punti su una curva ellittica e li “offusca” con una chiave casuale sua. Invia questi valori offuscati al mittente.
2. Il mittente applica una propria chiave segreta a tutti i valori ricevuti e li rimanda al destinatario, insieme a una *prova crittografica* che attesta di aver usato la stessa chiave per tutti (prova zero-knowledge).
3. Il destinatario rimuove il proprio offuscamento iniziale. Ora ha i valori cifrati solo sotto la chiave del mittente.
4. Il mittente invia anche le voci della propria rubrica, ugualmente cifrate con la propria chiave. Il destinatario confronta e trova le corrispondenze.

In tutto questo processo nessuno dei due ha mai visto in chiaro i dati dell’altro: il confronto avviene interamente su valori crittografici [1].

8.3.4 Protezione contro input malevoli

Un possibile abuso del sistema sarebbe quello di un attaccante che inserisce deliberatamente nella propria rubrica il numero di una persona famosa (un politico, un CEO) per cercare di carpire i suoi identificatori tramite la PSI.

PrivateDrop previene questo attacco richiedendo che gli input al protocollo siano *verificati da Apple*: il dispositivo può usare come input PSI solo gli identificatori che Apple ha già certificato come appartenenti all’account iCloud dell’utente. Un numero non verificato viene semplicemente ignorato prima ancora di entrare nel protocollo [1].

8.4 Ottimizzazioni di performance

8.4.1 Precomputazione

Le operazioni crittografiche più pesanti vengono eseguite in anticipo, quando il dispositivo è in carica e inattivo. In questo modo, quando l'utente apre il pannello di condivisione, la maggior parte del lavoro è già fatta e il ritardo percepito è minimo. I valori precomputati vengono aggiornati automaticamente solo quando la rubrica cambia [1].

8.4.2 Riduzione dei messaggi

Le due esecuzioni PSI (DO2 e DO3) vengono ottimizzate combinando insieme alcuni messaggi: il secondo messaggio della prima esecuzione viene inviato insieme al primo messaggio della seconda esecuzione in un unico pacchetto. In questo modo i round di comunicazione totali scendono da quattro a due, dimezzando le latenze di rete [1].

8.4.3 Padding degli input

Per evitare che la dimensione del messaggio riveli quante voci ha la rubrica di ciascuno, gli insiemi vengono sempre riempiti con elementi fittizi fino a una dimensione fissa: 10 000 voci per la rubrica e 10 per gli identificatori. La probabilità che un elemento fittizio produca una falsa corrispondenza è inferiore a 2^{-40} , ossia trascurabile [1].

8.5 Valutazione delle performance

I test sperimentali su iPhone e MacBook reali mostrano che PrivateDrop aggiunge circa 200 ms rispetto al protocollo originale, un ritardo impercettibile per l'utente:

Il collo di bottiglia non è la crittografia, che richiede meno di 50 ms, ma la latenza di rete tra i due dispositivi. La precomputazione riduce il ritardo totale a soli 400 ms, meno di mezzo secondo [1].

Scenario	AirDrop originale	PrivateDrop
Senza precomputazione	~200 ms	~800 ms
Con precomputazione	~200 ms	~400 ms
Rubrica > 10 000 voci	~200 ms	< 1 000 ms

Tabella 8.2: Confronto dei ritardi di autenticazione [1]

8.6 Integrazione nel protocollo AirDrop

PrivateDrop modifica solo la fase di autenticazione di AirDrop. Invece di trasmettere subito il validation record in chiaro, il messaggio `HTTPS POST /Discover` avvia le due esecuzioni PSI. Il validation record viene condiviso solo al termine, quando entrambi i dispositivi hanno ottenuto la prova crittografica di conoscersi a vicenda.

Le fasi di discovery BLE e di trasferimento file rimangono completamente invariate. Questo garantisce la compatibilità con i dispositivi che non implementano PrivateDrop: se uno dei due non supporta il nuovo protocollo, la connessione cade comunque in modo sicuro [1].

8.7 Risposta di Apple

Heinrich et al. hanno comunicato responsabilmente le vulnerabilità ad Apple Product Security prima della pubblicazione, come previsto dalle buone pratiche di *responsible disclosure*. Il lavoro è stato presentato alla USENIX Security 2021 e il codice è disponibile pubblicamente su GitHub [10].

Nonostante questo, Apple non ha integrato PrivateDrop nelle versioni pubbliche di iOS e macOS. A distanza di anni dalla divulgazione, AirDrop continua a usare hash SHA-256 senza salt nelle versioni distribuite agli utenti [1]. Questa mancata risposta è rilevante non solo dal punto di vista tecnico, ma anche etico: milioni di dispositivi rimangono esposti a vulnerabilità documentate e con soluzione disponibile.

CAPITOLO 9

Conclusioni

Questo elaborato ha analizzato in profondità la vulnerabilità di Contact Discovery del protocollo AirDrop, mostrando come un errore crittografico apparentemente semplice — l'uso di hash SHA-256 senza salt su dati a bassa entropia come i numeri di telefono — apra la porta a tre scenari di attacco distinti e concreti: *Sender Leakage*, *Receiver Leakage* e *Log File Leakage*.

La radice del problema non risiede in un bug implementativo, ma in una scelta progettuale sbagliata: trattare una funzione di hash come meccanismo di protezione dell'identità, senza considerare che lo spazio dei numeri di telefono è sufficientemente piccolo da rendere praticabile la precomputazione tramite rainbow table in pochi minuti su hardware comune.

La verifica sperimentale condotta nel Capitolo 7, tramite il framework OpenDrop su dispositivi reali, ha confermato empiricamente quanto descritto in letteratura. In particolare, è stato dimostrato che la sequenza `/Discover` → `/Ask` → `/Upload` avviene ancora in iOS 16/17, e che il protocollo trasmette in chiaro il `SenderPushToken`, un identificatore univoco del dispositivo non documentato come vettore di rischio nei paper originali. È stato inoltre verificato che il modello di attacco si è evoluto da uno scenario di *eavesdropping* passivo (iOS 14/15) a uno scenario di **honeypot attivo** (iOS 16/17), in cui la vulnerabilità crittografica sottostante rimane intatta pur

richiedendo ora un'interazione della vittima.

La soluzione esiste ed è stata proposta e validata sperimentalmente: *PrivateDrop*, basato su *Private Set Intersection* (PSI), elimina completamente la necessità di trasmettere hash invertibili, completando l'autenticazione reciproca in meno di un secondo senza degradare l'esperienza utente [1].

La risposta di Apple, tuttavia, è stata emblematica. Nel novembre 2022, con iOS 16.1.1, Apple ha introdotto un limite di 10 minuti sulla modalità “Tutti” di AirDrop — inizialmente *solo* per i dispositivi venduti in Cina, in risposta alle pressioni del governo cinese dopo che la funzione era stata usata dai manifestanti contro il governo di Xi Jinping. A seguito delle polemiche internazionali su questa scelta politicamente motivata, la stessa restrizione è stata estesa a *tutti gli utenti globali* con iOS 16.2 nel dicembre 2022, giustificandola stavolta come misura anti-spam.

Questa modifica, però, non tocca in alcun modo il meccanismo crittografico vulnerabile: agisce sul tempo di esposizione della modalità “Tutti”, non sugli hash SHA-256 privi di salt. Tant'è che nel gennaio 2024 le autorità cinesi hanno comunque identificato i dissidenti sfruttando esattamente il *Log File Leakage* descritto nel Capitolo 3, invertendo gli hash parziali estratti dai file di log di `sysdiagnose` tramite rainbow table.

Il caso AirDrop rappresenta quindi un esempio concreto di come le decisioni aziendali sui prodotti non seguano sempre la logica tecnica della sicurezza: la patch crittografica esiste, è pubblica, è validata sperimentalmente e — come dimostrato anche dalla nostra verifica pratica — la vulnerabilità rimane sfruttabile nelle versioni più recenti di iOS. La finestra di esposizione rimane aperta per scelta, non per impossibilità tecnica. Come insegna il ciclo di vita delle vulnerabilità studiato nel corso, finché una patch non viene distribuita e applicata su tutti i sistemi, la vulnerabilità rimane attiva — indipendentemente da quanto tempo sia nota pubblicamente.

Bibliografia

- [1] A. Heinrich, M. Hollick, T. Schneider, M. Stute e C. Weinert, “PrivateDrop: Practical Privacy-Preserving Authentication for Apple AirDrop,” in *30th USENIX Security Symposium (USENIX Security '21)*, USENIX Association, 2021, pp. 3577–3594. indirizzo: <https://www.usenix.org/conference/usenixsecurity21/presentation/heinrich>
- [2] A. Heinrich, M. Hollick, T. Schneider, M. Stute e C. Weinert, “DEMO: AirCollect: Efficiently Recovering Hashed Phone Numbers Leaked via Apple AirDrop,” in *14th ACM Conference on Security and Privacy in Wireless and Mobile Networks (WiSec '21)*, ACM, 2021, pp. 371–373. DOI: 10.1145/3448300.3468252
- [3] M. Stute, D. Kreitschmann e M. Hollick, “One Billion Apple’s Secret Sauce: Recipe for the Apple Wireless Direct Link Ad Hoc Protocol,” in *International Conference on Mobile Computing and Networking (MobiCom)*, ACM, 2018. DOI: 10.1145/3241539.3241566
- [4] National Institute of Standards and Technology, “Secure Hash Standard (SHS),” NIST, rapp. tecn. FIPS PUB 180-4, 2015. DOI: 10.6028/NIST.FIPS.180-4
- [5] P. Oechslin, “Making a Faster Cryptanalytic Time-Memory Trade-Off,” in *Advances in Cryptology – CRYPTO 2003*, Springer, 2003, pp. 617–630. DOI: 10.1007/978-3-540-45146-4_36

- [6] International Telecommunication Union, *The International Public Telecommunication Numbering Plan*, ITU-T Recommendation E.164, 2010. indirizzo: <https://www.itu.int/rec/T-REC-E.164>
- [7] National Institute of Standards and Technology, "Recommendation for Password-Based Key Derivation," NIST, rapp. tecn. SP 800-132, 2017. DOI: 10.6028/NIST.SP.800-132
- [8] MITRE Corporation, *Common Weakness Enumeration (CWE)*, <https://cwe.mitre.org>, 2023.
- [9] M. Heinrich Alexander; Stute, *opendrop*, 2021. indirizzo: <https://github.com/seemoo-lab/opendrop>
- [10] M. Heinrich Alexander; Stute, *privatedrop*, 2021. indirizzo: <https://github.com/seemoo-lab/privatedrop>